LiteTest Contributor Guide

Document Version: 1.0

Runner Version: 1.0.0

Test Version: 1.0.0

This document defines the contribution process, coding standards, documentation rules, and versioning guidelines for LiteTest.

Preface

This document is intended for LiteTest contributors who need guidance on enhancing and maintaining LiteTest.

For a list of other LiteTest documents and the repository layout, see the LiteTest Documentation Guide (`LiteTest_Documentation_Guide.md`).

For a glossary of terms, see the LiteTest Glossary Reference (`LiteTest_Glossary_Reference.md`).

For a glossary of general LiteTest terms, see the LiteTest Glossary Reference document. For contributor-Sspecific terms: see the Glossary appendix at end of this document.

Document Version History

Document	Runner	Test	Date	Comment	Author/Editor
1.0	1.0.0	1.0.0	2026-06-11	Initial version.	Paul Sinclair

- The **Document** column records the document's version with the format `M.u` (Major, update).
- The **Runner** column records the LiteTest Runner API version current at the time this document version was published and is defined by its `LT_RUNNER_VERSION` macro.
- The **Test** column records the LiteTest Test API version current at the time this document version was published and is defined by its `LT_TEST_VERSION` macro.
- Both Runner and Test use the version format `"M.m.p"` (Major, minor, patch).
- `M` is the same for the Document, Runner, and Test versions.

The document's update version tracks updates to this document and does not correspond to a minor or patch version. `u` increments whenever this document is updated without a change to `M`, and it resets to `0` when `M` is incremented.

Table of Contents

1. Introduction

This Contributor Guide defines the expectations and rules for contributing to LiteTest. It covers versioning, testing, documentation, code style, and pull request requirements.

Contributors should read this document before submitting changes to ensure consistency across the LiteTest codebase and documentation.

2. Versioning Rules

LiteTest uses semantic versioning: - For .h and .c files: `M.m.p` (major, minor, patch). - For .md files: `M.u` (major, update).

When to increment:

- Update:
  - Document updated.
- Patch:
  - Bug fixes or implementation improvements.
- Minor:
  - Minor additions to the APIs (e.g., new function or macro)
  - Minor additions to the LiteTest framework.
  - No breaking changes (syntax or behavior) to existing public APIs.
  - No breakind changes to the LiteTest framework
  - Minor refactorying of implementation.
- Major
  - Major additions to the LiteTest APIs.
  - Major additions to the LiteTest framework.
  - Significant refactoring of implementation.
  - The following require incrementing the Major version but should be avoided by having opt-in or other mechanism to maintain compatibility:
    - Breaking changes (syntax or behavior) to existing public APIs.
    - Breaking changes to the LiteTest framework.
  - If incremented, it must be updated for all files (.h, .c, and .md) to the same major value, even if the file was not otherwise modified or updated with update, minor, and patch reset to 0.

When updating the version for the Runner API: - Update `LT_RUNNER_VERSION` in `litetest_runner.h`. - Update `LT_RUNNER_VERSION_C` in `litetest_runner.c`.

When updating the version for the Test API: - Update `LT_TEST_VERSION` in `litetest_test.h`. - Update `LT_TEST_VERSION_C` in `litetest_test.c`.

When updating the version for a document: - Add a new new entry to the top of the Document Verion History table for the document, with an incremented value for u if major is not incremented, otherwise with with 0.

API compatibility rules: - Public APIs are additive by default. - Do not change existing signatures. - Do not change return code meanings. - Deprecate before removing. - Provide migration guidance for major changes including how to opt-in for new features and behavior.

3. Testing Requirements

- Build and run tests from the repository root:

```
make run
```

- Ensure report formatting changes are reflected in documentation examples.
- Keep test code aligned with the current version of `litetest_runner.h`.

4. Documentation Rules

- `README.md` must remain short and onboarding-focused.
- A User Guide contains conceptual explanations and examples.
- A API Reference contains public API definitions only.
- An internal guide or reference must not leak into public docs.
- Update documentation when enhancing macros, behavior, or report format.

5. Code Style

- C99.
- POSIX.1-2001 APIs only.
- Keep `litetest_runner.h` self-contained.
- Keep `litetest_runner.c` implementation-only.
- Avoid intermixing test code with helper logic inside test group functions.

6. Writing Style Consistency

(See the full style rules in the Documentation Style Guide above.)

This section exists here to ensure contributors do not overlook the requirement for parallel structure and consistent writing patterns across all LiteTest documentation in all types of files (e,g., ,md, .h, .c, .yml, ,sh, Makefile, etc.).

6.1 Documentation Style Guide

1. Tone

- Technical, precise, and neutral.
- No marketing language.
- Prefer clarity over cleverness.

2. Formatting

- Use backticks for code identifiers.
- Use fenced code blocks for file trees, examples, and commands.
- Keep line lengths reasonable for GitHub rendering.

3. Writing Guidelines

- Define terms once, and then use them consistently.
- Avoid synonyms for technical concepts (e.g., always “update version,” never “revision”).
- Keep paragraphs short.
- Use lists for enumerations.

4. `Click to view`

- Use `Click to view` sections and subsections to keep the document readable while still accommodating large amounts of technical detail:
  - Collapsing sections allows readers to scan the structure and expand only what they need.
  - This keeps the document manageable, avoids overwhelming readers with unrelated detail, and makes the document easier to navigate.

5. Writing Style Consistency To keep LiteTest documentation clear and easy to read, maintain **parallel structure** within lists and related sentences. In practice:

- Start list items with the same part of speech (typically a verb).
- Keep grammatical patterns consistent across bullets.
- Avoid mixing styles such as “Keep paragraphs short” with “Using lists for enumerations.”
- Rewrite items as needed so the list reads smoothly and uniformly.

This guideline applies to all LiteTest documentation (.md files).

7. Pull Requests

Before submitting a PR:

- Ensure tests pass.
- Update version numbers if needed.
- Update documentation as needed.
- Keep changes focused and well-scoped.

Glossary

For general LiteTest terms, see the LiteTest Glossary Reference document.

Contributor-Specific Terms:

- **Approver**: A contributor with commit access who reviews pull requests, enforces versioning rules, and ensures documentation consistency.
- **Breaking Change**: A change that alters public API behavior, removes or renames macros, changes return semantics, or requires a major version bump.
- **Contributor**: A person submitting code, documentation, fixes, or improvements to LiteTest.
- **Deprecation**: A public API element marked for removal in a future major version, requiring documentation updates and migration guidance.
- **Documentation Update**: Any change to released LiteTest documentation requires incrementing the update version or, if an API major version is incremented, incrementing the major version and resetting the update version to 0.
- **Internal API Change**: A change to an API's internals that does not affect public API users but may require updates to an Internal Guide or Internal Reference.
- **Major Increment**: A structural or conceptual overhaul of LiteTest that causes a breaking change.
- **Public API Change**: Any modification to the LiteTest Runner or Test API that requires a version bumpv.
- **Pull Request Scope**: A guideline requiring PRs to be focused, minimal, logically grouped, and not mixing unrelated changes.
- **Test Coverage Requirement**: The expectation that all changes to LiteTest include new tests (if adding behavior), updated tests (if modifying behavior), and no regressions.